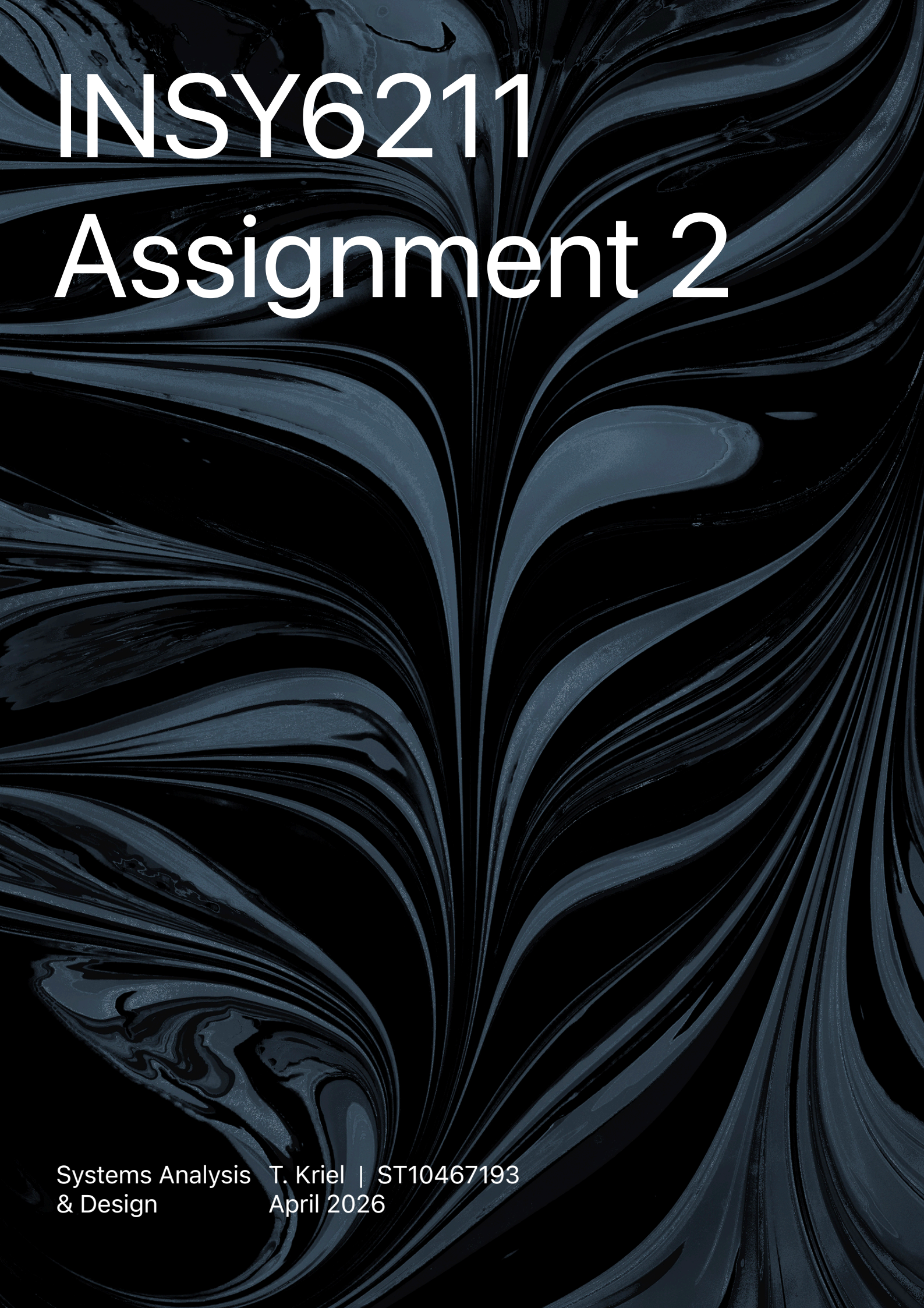




INSY6211

Assignment 2

Systems Analysis & Design T. Kriel | ST10467193
April 2026

Index.

QUESTION 1

System Vision Document

A System Vision Document (SVD) is a short, approval-oriented artefact used to define the scope of the proposed system by articulating the business problem it addresses, the capabilities it will deliver, and the measurable benefits it is expected to produce [1]. The extended SVD presented below follows the three-component structure prescribed by Satzinger, Jackson, and Burd [1] and is augmented with project scope, stakeholders, and assumptions/constraints consistent with broader requirements-engineering and systems-analysis practice [2], [4].

Problem/Opportunity Description

The airport has recently upgraded its infrastructure to accommodate international flights and is anticipating a sustained increase in passenger volume, particularly inbound tourists who are unfamiliar with the facility. Empirical research on airport service quality demonstrates that way-finding difficulty, ambiguous flight information, and slow response during medical or safety incidents are among the strongest drivers of passenger dissatisfaction and negative online reviews [3]. Without a coordinated digital channel, customer relations will rely on physical information desks and static signage, both of which scale poorly with international traffic, which consequently creates a material risk to passenger satisfaction and to the airport's brand as an international destination.

The opportunity is to deliver a Customer Relations Management System (CRMS) that centralises real-time flight and amenity information, in-terminal navigation, and emergency medical dispatch into a single, multi-channel platform accessible through permanently installed kiosks and a downloadable mobile application. The CRMS will address the customer-relations gap introduced by international expansion while positioning the airport as a top-of-class service provider.

System Capabilities

The CRMS will provide the following capabilities; detailed requirements will be specified in subsequent documents [1]:

- ◆ **Flight information services:** real-time display of arrivals, departures, gate allocations, delays, and boarding status, with push notifications to registered mobile users.
- ◆ **Amenity and service information:** directory of restaurants, retail outlets, lounges, restrooms, and support services (currency exchange, baggage services, car rental) with hours, location, and contact details.
- ◆ **Indoor and outdoor navigation:** GPS- and beacon-assisted turn-by-turn directions to boarding gates, amenities, and ground-transport links, with accessibility routing options.
- ◆ **Emergency medical assistance:** a one-tap function that dispatches airport medical and ambulatory services to the caller's current location, including automatic transmission of GPS coordinates.

- ◆ **Multi-channel delivery:** a browser-based kiosk front-end deployed across the airport and a downloadable iOS/Android mobile application that shares a common back-end.
- ◆ **Multilingual and accessibility support:** localisation for major international traveller languages and WCAG-aligned accessibility features, consistent with inclusive-design practice in contemporary software engineering [4].

Business Benefits

The capabilities above are expected to produce the following measurable benefits for the airport [1]:

- ◆ **Improved passenger satisfaction and online ratings,** directly addressing the service-quality drivers identified in airport-specific empirical research [3].
- ◆ **Reduced workload on physical information desks,** allowing existing staff to focus on high-touch assistance and lowering per-passenger service cost.
- ◆ **Faster emergency response,** with the potential to reduce adverse health outcomes and limit reputational and legal exposure from incidents.
- ◆ **Competitive differentiation** as an internationally competitive airport, supporting the strategic goal of attracting and retaining tourist traffic.
- ◆ **Data-driven service improvement,** since CRMS usage data (popular amenities, navigation friction points, emergency hotspots) will feed continuous operational planning and future CRM analytics.

Project Scope

In scope: kiosk software, iOS and Android mobile applications, a shared cloud back-end, integration with airline flight-data feeds, integration with the airport medical dispatcher, and indoor/outdoor navigation services.

Out of scope: airline ticketing and check-in, baggage-tracking systems, immigration and border-control systems, and physical hardware procurement (handled under a separate infrastructure work-stream). A clearly demarcated scope at the outset is essential because uncontrolled scope expansion is a dominant cause of project failure in systems-analysis practice [2].

Key Stakeholders

The principal stakeholders are:

- ◆ Airport executive management (project sponsor)
- ◆ International and domestic passengers (primary end-users)
- ◆ On-airport medical and ambulatory service providers
- ◆ Airline partners supplying flight data

- ◆ Concession vendors and amenity providers
- ◆ Airport operations and maintenance personnel
- ◆ The software development team
- ◆ Aviation regulatory and data-protection authorities

Early stakeholder identification supports the elicitation and negotiation activities that dominate the requirements phase and reduced the risk of missing a critical viewpoint [4].

Assumptions and Constraints

Assumptions:

- ◆ Airport-wide WiFi and cellular coverage are sufficient for real-time kiosk and mobile operation.
- ◆ Airlines will expose real-time flight status through a documented API or data feed.
- ◆ On-airport medical services can be contacted digitally on a 24/7 basis.
- ◆ The majority of passengers carry a smartphone; kiosks serve those who do not.

Constraints:

- ◆ **Reliability:** the system must be “top of its class” with no tolerance for operational failure, which will drive downstream non-functional requirements and architectural decisions.
- ◆ **Scope volatility:** a key executive sponsor is known to revise requirements frequently; this argues for an adaptive, iterative delivery approach that can absorb change without destabilising the schedule [2], [4].
- ◆ **Schedule incentive:** a bonus is offered for early delivery, but the “no room for failure” requirement means quality cannot be traded off against speed.
- ◆ **Regulatory compliance:** international passenger data imposes data-protection obligations (e.g., POPIA in South Africa; GDPR for EU travellers) as well as aviation-sector privacy and safety rules [4].

QUESTION 2

Q 2.1

Functional requirements describe the activities that the system must perform to support its users' work - the business functions and use cases it implements [1]. In the FURPS framework adopted by Satzinger, Jackson, and Burd, functional requirements are the "F" category, distinct from the usability, reliability, performance and security characteristics that describe how the system behaves [1]. Each requirement below is expressed in the imperative "shall" form recommended by mainstream software-engineering practice [4], is traceable to a specific capability identified in the System Visions Document (Q 1.1), and includes acceptance criteria so that the requirement is testable [2], [4].

I. Real-time flight information

Requirement:

The CRMS shall present passengers with real-time flight information for all departures and arrivals, including flight number, airline, origin/destination, scheduled and actual times, gate allocation, and boarding status. The system shall push notifications to registered mobile users within 30 seconds of any status change (delay, gate reallocation, cancellation, or boarding call).

Justification:

This requirement realises the "Information" capability mandated in the scenario. Passengers cite ambiguous or delayed flight information as a leading source of dissatisfaction in airport-specific service-quality studies, and timely digital updates are one of the strongest mitigations when irregular operations occur [3]. Satzinger et al. emphasise that functional requirements should be tied directly to the business-uses identified during scoping, and real-time flight status is the most frequent interaction passengers will have with the system [1].

II. Indoor navigation and amenity location

Requirement:

The CRMS shall allow a passenger to select a single amenity (restaurant, retail outlet, restroom, lounge, information desk, or boarding gate) from a searchable directory and shall compute and display turn-by-turn walking directions from the passenger's current location to that amenity, using GPS positioning outdoors and beacon-assisted indoor positioning inside the terminal. Directions shall update dynamically as the passenger moves, and the system shall support accessibility routing (step-free paths) on request.

Justification:

This requirement realises the "Navigation capability" in the scenario and is bounded by the Q3 business rule that a passenger may request the location of only one amenity at a time, which is deliberately taken into consideration now so that the later data model and use-case descriptions remain consistent. Way-finding difficulty is one of the empirically documented drivers of passenger

dissatisfaction at large international airports [3], and Satzinger et al. note that functional requirements should capture not only the core action but also the business rules that constrain it [1].

III. Emergency medical assistance request

Requirement:

The CRMS shall provide a single-action ("one-tap") function through which a passenger can request medical and ambulatory assistance. On activation, the system shall (i) transmit the passenger's current geographic coordinates and identifier to the airport medical dispatcher, (ii) create a time-stamped incident record for audit purposes, and (iii) return an acknowledgement to the passenger within 5 seconds. The function shall be available from both the kiosk and mobile application front-ends.

Justification:

This requirement realises the third explicit capability in the scenario — instant connection to medical and ambulatory services. Because each emergency is a discrete event and the same passenger may experience multiple emergencies (per the Q3 business rules), the requirement is framed at the level of a single request; this aligns with the guidance that functional requirements should reflect the underlying procedures and rules that govern business behaviour [1]. The low acknowledgement latency and the audit record together support downstream reliability and traceability obligations that a safety-critical health function must satisfy under contemporary software-engineering practice [4].

Q 2.2

Non-functional requirements describe characteristics of the system other than the activities it performs — how well the system must behave, rather than what it must do [1]. Satzinger et al.'s FURPS framework groups these characteristics into Usability, Reliability, Performance, and Security [1], and the international standard ISO/IEC 25010 provides a broader, widely adopted taxonomy (Functional Suitability, Performance Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability) against which specific targets can be calibrated [5]. The two requirements below are chosen because they reflect the two strongest constraints stated in the scenario: the executive mandate that the system be "top of its class" with "no room for failure," and the introduction of international passengers together with sensitive medical data.

I. Reliability (availability and fault tolerance)

Requirement:

The CRMS shall achieve a minimum availability of 99.95% per calendar month across all public-facing front-ends (kiosks and mobile application), measured excluding scheduled maintenance windows published at least 48 hours in advance. The medical-emergency request function specifically shall have no single point of failure: it shall remain operational under the loss of any single server node, single network link, or single availability zone, and shall fail over within 30

seconds. Mean Time to Recovery (MTTR) for any outage affecting the emergency function shall not exceed 5 minutes.

Justification:

Reliability requirements describe the dependability of a system — how often it exhibits service outages or incorrect processing, and how quickly it detects and recovers from them [1]. The scenario is explicit that there is "no room for failure" and that the system must be "top of its class," which makes a quantified availability target essential rather than aspirational. ISO/IEC 25010 decomposes reliability into availability, fault tolerance, and recoverability, each of which maps directly to a clause of the requirement above and provides the standardised vocabulary used in acceptance testing [5]. Sommerville further notes that for systems whose failure can lead to physical harm, reliability constraints tend to dominate architectural decisions and must be specified with measurable targets rather than qualitative adjectives [4] — a point that is doubly relevant here because the emergency-medical function is embedded in the same system as the information and navigation functions.

II. Security (confidentiality, integrity, and access control)

Requirement:

The CRMS shall protect all passenger data — including identity, location history, and medical-emergency records — in accordance with contemporary information-security practice. Specifically:

- (i) all network traffic between front-ends and the back-end shall be encrypted using TLS 1.3 or later;
- (ii) passenger data at rest shall be encrypted with AES-256;
- (iii) access to administrative and dispatcher functions shall require multi-factor authentication;
- (iv) every access to a medical-emergency record shall be logged with user, time, and purpose, and logs shall be tamper-evident; and
- (v) the system shall comply with applicable data-protection legislation, including the South African Protection of Personal Information Act (POPIA) and, where EU passengers are involved, the General Data Protection Regulation (GDPR).

Justification:

Security requirements in the FURPS framework describe how access to the application is controlled and how data is protected during storage and transmission [1]. The CRMS handles three categories of sensitive data — personal identity, real-time location, and medical incident information — whose combined exposure would be seriously harmful; this places security on the same footing as reliability for this project. ISO/IEC 25010 decomposes security into confidentiality, integrity, non-repudiation, accountability, and authenticity, all of which are represented in the clauses above [5]. Sommerville identifies security requirements as a class of non-functional requirement whose specification must be quantitative and enforceable rather than aspirational, particularly for systems that process personal and health-related data across international jurisdictions [4].

QUESTION 3

Q 3.1

The three business rules describe four entities - **Passenger**, **Amenity**, **Flight Booking**, and **Medical Emergency** - with Passenger as the "central actor" that participates in every relationship [1]. In line with Satzinger, Jackson, and Burd's guidance on domain modelling, each entity is given a single primary key (PK), each relationship is labelled with a verb phrase, and every many-side association uses a foreign key (FK) to its parent entity [1]. Dennis, Wixom, and Roth stress the same conventions, adding that entity names should be singular nouns and attribute names should be concise and meaningful [2].

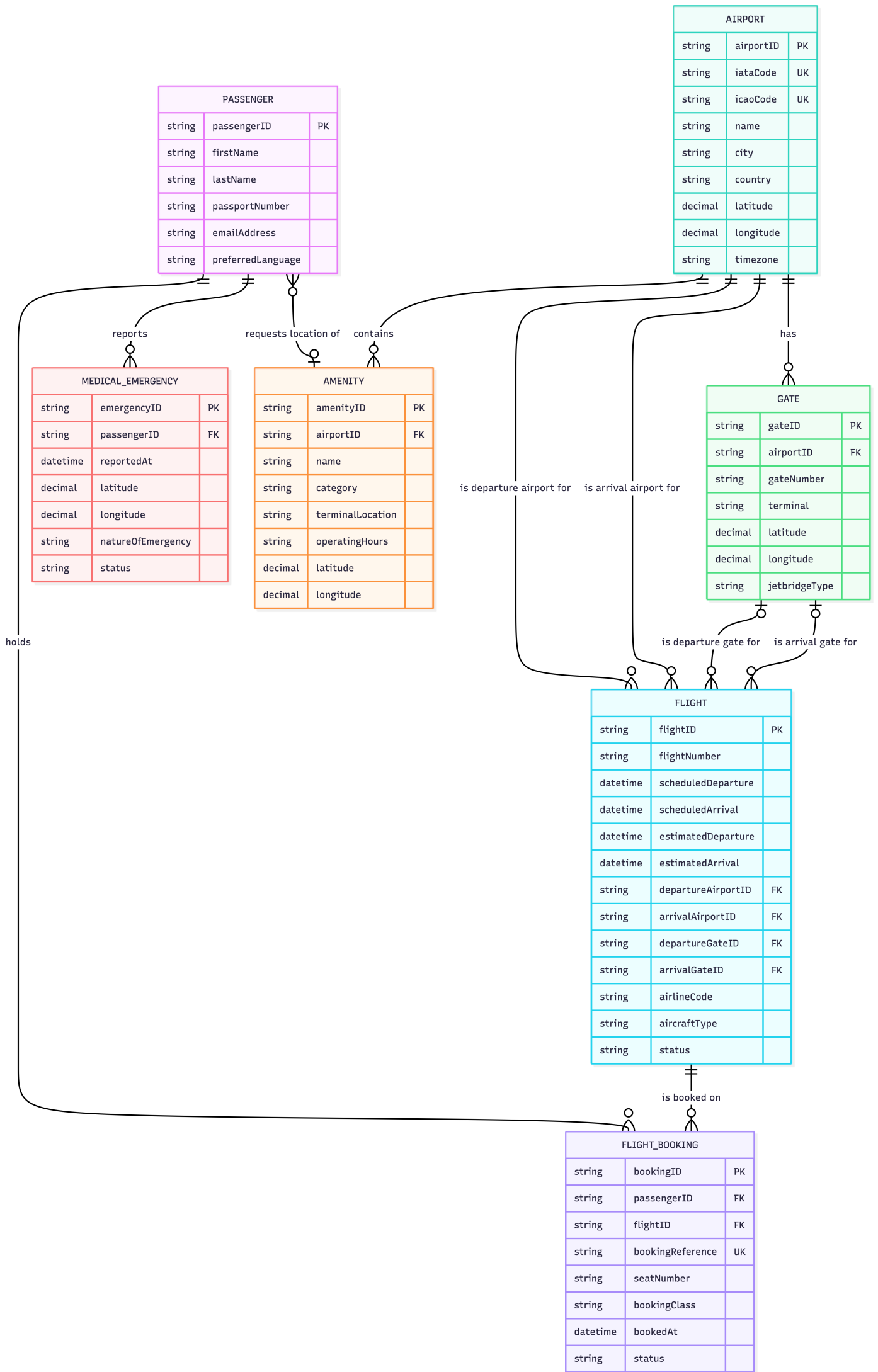
A note on one of the business rules before the diagram: "One passenger can request the location of one and only one amenity at a time" has been modelled as **0..* passengers ↔ 0..1 amenity**. The phrase "at a time" makes it temporally bounded (a passenger is either actively looking for something, or they are not, thus **0..1** is modelled on the passenger side), while "one and only one" forbids requesting multiple amenities simultaneously [1]. Modelling this as a mandatory **1..1** on the passenger side would incorrectly force every passenger to always be making a request, which does not reflect the domain. The final ERD has been rendered using Mermaid syntax, which will include a reader's guide below, as well as the original code for generating the ERD, with in-line comments.

NOTE: I have decided to implement a revised version of the original ERD scope expanded upon above, with the following changes and implementations:

The revised model promotes two new entities: **FLIGHT** and **AIRPORT**, and repurposes **FLIGHT_BOOKING** as an associative (join) entity between a **PASSENGER** and a specific **FLIGHT** instance, following the standard many-to-many resolution pattern described by Satzinger, Jackson, and Burd [1] and reinforced by Dennis, Wixom, and Roth [2]. **GATE** is promoted to its own entity because it carries attributes (terminal, coordinates, jetbridge type) and relationships (to airport; to flight) of its own; **TERMINAL** is retained as a string attribute on **GATE** rather than promoted, as promoting every noun in the domain inflates the schema without adding modelling value [1]. All primary keys use surrogate identifiers so that business keys (flight number, IATA code) remain free to evolve and be reused across time, which particularly matters for **FLIGHT**, because flight numbers like "SA203" are reused every operating day, and without a surrogate key the model could not represent more than one instance of the same numbered flight [2].

Summary of changes and design justifications

Change	Rationale	Source
Promoted FLIGHT to its own entity	The original model conflated a passenger's booking with the flight itself. Separating them lets many passengers book the same flight, which is the real-world cardinality	[1],[2]
Promoted AIRPORT to its own entity	Avoids duplicating airport metadata across every flight and every amenity; enables multi-airport extension later	[1],[2]
Promoted GATE to its own entity	Gates have their own attributes (terminal, coordinates) and independent relationships (to airport, to flights); meets the "attributes and relationships of its own" test for entity promotion	[1]
Kept TERMINAL as an attribute of GATE	A would-be TERMINAL entity would carry only a name and an airport FK - no independent attributes; adding entities purely because the domain has a noun for them is unnecessary	[1]
Used <i>surrogate PKs everywhere</i> (flightID , airportID , etc.)	Business keys like flightNumber and IATA codes are reused, reassigned, or occasionally changed by industry bodies; surrogate keys insulate the model from these changes	[2]
Added <i>unique business keys</i> (passportNumber , iataCode , icaoCode , bookingReference)	Marked as 'UK' in the diagram - these are what people look up by and must be unforced as unique, but they are NOT the PK	[1],[2]
FLIGHT_BOOKING treated as an associative entity	The M:N relationship between PASSENGER and FLIGHT is resolved through a join entity that carries its own attributes (seat, PNR, status)	[1],[2]
Did not denormalise airport/time data onto FLIGHT_BOOKING	Normalised to 3NF; all booking information about the flight is reachable via the flightID FK; denormalisation for historical snapshots is an engineering decision taken at the physical-database or data-warehouse layer, not in the conceptual ERD	[1],[2]
Modelled two distinct FLIGHT ↔ AIRPORT relationships (departure and arrival)	Same pair of entities can legitimately have multiple relationships; the verb phrases disambiguate them	[1]
Flight-to-gate relationships marked <i>optional on the flight side</i> ('o')	Reflects real operational practice: gate assignments are published closer to the flight's departure rather than at schedule creation	



Summary of Keys

Entity(s)	Key(s)
Primary Keys	
PASSENGER	passengerID
AMENITY	amenityID
FLIGHT_BOOKING	bookingID
MEDICAL_EMERGENCY	emergencyID
FLIGHT	flightID
GATE	gateID
AIRPORT	airportID
Foreign Keys	
FLIGHT → AIRPORT	departureAirportID; arrivalAirportID
FLIGHT → GATE	departureGateID; arrivalGateID
FLIGHT_BOOKING → PASSENGER	passengerID
FLIGHT_BOOKING → FLIGHT	flightID
GATE → AIRPORT	airportID
AMENITY → AIRPORT	airportID
MEDICAL_EMERGENCY → PASSENGER	passengerID
Unique Keys	
AIRPORT	iataCode; icaoCode
FLIGHT_BOOKING	bookingReference

How to read Mermaid ERD Syntax - Reader's Guide

Mermaid is a Javascript-based diagramming library that renders diagrams from plain-text definitions, and its ER-diagram syntax was designed as a text-authored equivalent of the crow's foot notation found in most systems-analysis textbooks [6]. The underlying modelling concepts - entities, attributes, primary keys, cardinalities - are unchanged from the traditional guidelines in Satzinger et al. and Dennis et al. [1], [2]. Mermaid helps to encode these traditional concepts into ASCII so they can be saved as Markdown files or source code that can be versioned in Git (see Github section). Mermaid ER Diagrams primarily have two kinds of statements: relationship statements (which connect two entities) and entity blocks (which list the attributes inside { }).

1. The outer wrapper

Every ER diagram begins with the keyword **erDiagram** [6]. Everything after it, up until the end of the code block, is part of the diagram. Lines starting with `%%` are comments and are ignored by the renderer.

2. Relationship lines

A relationship line adheres to the following template:

```
ENTITY_A <left-glyph>--<right-glyph> ENTITY_B : "verb phrase"
```

The two glyphs are small ASCII pictograms borrowed from crow's-foot notation. The glyph closest to an entity describes that entity's participation in the relationship, read from the opposite entity's point of view [6]. Here is an example with explanations:

```
PASSENGER ||-o{ FLIGHT_BOOKING
```

- The `||` sits next to `PASSENGER`, so from a booking's perspective there is **exactly one** passenger per booking.
- The `o{` sits next to `FLIGHT_BOOKING`, so from a passenger's perspective there are **zero or many** bookings per passenger.
- In summary: a passenger holds **zero or more** flight bookings; every booking is held by **exactly one** passenger.

3. Glyph Table

Value (left)	Value (Right)	Meaning
o	o	Zero or one
		Exactly one
}o	o{	Zero or more (no upper limit)
}	{	One or more (no upper limit)

The `-` in the middle is a solid line, denoting an identifying relationship; `..` produces a dashed line for non-identifying relationships [6].

4. Entity Blocks

An entity block declares attributes inside curly braces, one per line:

```
ENTITY_NAME {  
  
<type> <attributeName> [PK | FK | UK] "optional comment"  
  
}
```

The type (e.g., string, int, datetime) is a label, not a constraint - there is no type-checking in Mermaid, so any vocabulary can be used that matches the target DBMS [6]. The PK, FK, and UK markers flag primary, foreign, and unique keys respectively and render as badges in the diagram [6]. Comments in double quotes render as tooltips in most Mermaid viewers.

Complete Mermaid syntax-code of ERD

NOTE: the version-controlled markdown-file of this can be found in the project Github repository as "erd.md" in **/diagrams**. The PNG export can also be found in **/diagrams** as "ERD.png".

```
erDiagram
    direction TB

    %% =====
    %% Airport Customer Relations Management System
    %% Entity Relationship Diagram - Q3.1
    %% =====

    %% ——— ENTITIES ———

    PASSENGER {
        string passengerID PK
        string firstName
        string lastName
        string passportNumber
        string emailAddress
        string preferredLanguage
    }

    AIRPORT {
        string airportID PK
        string iataCode UK
        string icaoCode UK
        string name
        string city
        string country
        decimal latitude
        decimal longitude
        string timezone
    }

    AMENITY {
        string amenityID PK
        string airportID FK
        string name
        string category
        string terminalLocation
        string operatingHours
        decimal latitude
        decimal longitude
    }
```



```

FLIGHT {
    string flightID PK
    string flightNumber
    datetime scheduledDeparture
    datetime scheduledArrival
    datetime estimatedDeparture
    datetime estimatedArrival
    string departureAirportID FK
    string arrivalAirportID FK
    string departureGateID FK
    string arrivalGateID FK
    string airlineCode
    string aircraftType
    string status
}

```

```

GATE {
    string gateID PK
    string airportID FK
    string gateNumber
    string terminal
    decimal latitude
    decimal longitude
    string jetbridgeType
}

```

```

FLIGHT_BOOKING {
    string bookingID PK
    string passengerID FK
    string flightID FK
    string bookingReference UK
    string seatNumber
    string bookingClass
    datetime bookedAt
    string status
}

```

```

MEDICAL_EMERGENCY {
    string emergencyID PK
    string passengerID FK
    datetime reportedAt
    decimal latitude
    decimal longitude
    string natureOfEmergency
    string status
}

```

%% ——— RELATIONSHIPS ———

```

%% Business Rule 1
PASSENGER}o--o|AMENITY:"requests location of"

```

```

%% Business Rule 2

```

```
PASSENGER| |-o{FLIGHT_BOOKING:"holds"

%% Business Rule 3
PASSENGER| |-o{MEDICAL_EMERGENCY:"reports"

%% Revised Additions
FLIGHT| |-o{FLIGHT_BOOKING:"is booked on"

%% Airport Relationships
AIRPORT| |-o{FLIGHT:"is departure airport for"
AIRPORT| |-o{FLIGHT:"is arrival airport for"
AIRPORT| |-o{AMENITY:"contains"
AIRPORT| |-o{GATE:"has"

%% Gate Relationships
GATE|o--o{FLIGHT:"is departure gate for"
GATE|o--o{FLIGHT:"is arrival gate for"
```

REFERENCES

- [1] J. W. Satzinger, R. B. Jackson, and S. D. Burd, *Systems Analysis and Design in a Changing World*, 7th edn. Boston, MA, USA: Cengage Learning, 2017.
- [2] A. Dennis, B. H. Wixom, and R. M. Roth, *Systems Analysis and Design*, 8th edn. Danvers, MA, USA: Wiley, 2021.
- [3] Fevzi Okumus et al., 'Airport service quality drivers of passenger satisfaction', *Tour. Rev.*, vol. 68, no. 4, pp. 3–18, Oct. 2013, doi: 10.1108/TR-09-2013-0047.
- [4] I. Sommerville, *Software Engineering*, 10th edn. Essex, England, U.K.: Pearson, 2016.
- [5] Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - Product quality model, *Product Quality Model ISO/IEC 25010:2023*, Geneva, Switzerland., Nov. 2023. [Online]. Available: <https://www.iso.org/standard/78176.html>
- [6] 'Entity Relationship Diagrams', Mermaid Documentation. Accessed: Apr. 23, 2026. [Online]. Available: <https://mermaid.js.org/syntax/entityRelationshipDiagram.html>